

Passo a passo do Serviço Python

Automação de Laudos Técnicos a partir de planilhas + normas + IA (OpenRouter) + PDF

Objetivo

Implementar um serviço backend em Python que receba planilhas, valide e normalize os dados, gere seções narrativas com LLM (apenas recomendações/justificativas), renderize um template HTML e gere o PDF final versionado, com persistência em Postgres/Supabase e arquivos no Supabase Storage.

Premissas

- Você já executou: `pip install -r requirements.txt` e está com o `.venv` ativo.
- O serviço usa FastAPI, arquitetura Hexagonal (Ports & Adapters), Postgres (Supabase) e Supabase Storage.
- O LLM entra somente para texto (JSON), nunca para validação/parse.

Visão geral e arquitetura

A arquitetura separa regras (domínio) de infraestrutura (DB, Storage, LLM, PDF). As rotas (API) são finas e chamam casos de uso (Application).

Camada	Responsabilidade	Pastas
API	HTTP, valida payload, chama Use Cases	app/api/routes
Application	Orquestra o fluxo (pipeline)	app/application/use_cases
Domain	Entidades, erros e Ports (interfaces)	app/domain/{entities,ports,schemas}
Adapters	Implementações (DB/Storage/LLM/PDF/Planilha)	app/adapters/*
Infrastructure	Config, logging, DI, engine DB	app/infrastructure/*

Convenções de paths no Storage

- uploads/{upload_id}/{filename}
- uploads/{upload_id}/clone_{YYYYMMDD}_{filename}
- reports/{report_id}/report_v{version}.pdf

Regra de ouro do pipeline

Validação e estrutura do documento são determinísticas. O LLM só gera texto e sempre em JSON. Qualquer saída não-JSON deve ser tratada como erro/retry.

Etapa 1 — Estrutura do projeto (hexagonal)

Objetivo: Criar pastas e arquivos base para API/Domain/Application/Adapters/Infrastructure. Sem implementar lógica ainda: apenas estrutura + placeholders.

Arquivos/Pastas impactados:

- app/api/main.py
- app/api/routes/{health.py, uploads.py, documents.py}
- app/domain/{entities, ports, schemas, errors.py}
- app/application/{use_cases, dto}
- app/adapters/{db, storage, llm, spreadsheet, pdf, norms}
- app/infrastructure/{config.py, logging.py, dependencies.py, db.py}
- tests/

Prompt para o agente (copiar e colar):

Você está em um projeto FastAPI Python já com venv e requirements instalados.
Crie a estrutura de pastas em /app seguindo arquitetura hexagonal:

```
- app/api/main.py
- app/api/routes/{health.py, uploads.py, documents.py}
- app/domain/{entities, ports, schemas, errors.py}
- app/application/{use_cases, dto}
- app/adapters/{db, storage, llm, spreadsheet, pdf, norms}
- app/infrastructure/{config.py, logging.py, dependencies.py, db.py}
```

Adicione `__init__.py` onde necessário.

Crie também `tests/` vazio.

Não escreva funcionalidades ainda: só a estrutura e arquivos “placeholder” com docstrings curtos.

Etapa 2 — Configuração (.env) e logging

Objetivo: Centralizar Settings (env vars), logging e base de injeção de dependências.

Arquivos/Pastas impactados:

- app/infrastructure/config.py
- app/infrastructure/logging.py
- app/infrastructure/dependencies.py
- .env.example

Prompt para o agente (copiar e colar):

Implemente configuração e logging:

```
1) app/infrastructure/config.py
- Use pydantic-settings (ou dotenv + pydantic) para ler variáveis do .env
- Exponha um objeto Settings com:
  ENV, API_PORT, DATABASE_URL,
  SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, SUPABASE_BUCKET,
  OPENROUTER_API_KEY, OPENROUTER_BASE_URL, LLM_MODEL

2) app/infrastructure/logging.py
- Configure loguru (ou logging) com nível por ENV
- Formato com timestamp, level, module, message

3) app/infrastructure/dependencies.py
- Funções get_settings(), get_logger()
- Placeholders get_db(), get_storage(), get_llm() retornando None por enquanto

Inclua exemplos mínimos e docstrings. Não implemente DB ainda.
```

Etapa 3 — Domain: entidades, erros e ports (contratos)

Objetivo: Definir o modelo de domínio e as interfaces que os adapters vão implementar. Isso congela o contrato e evita acoplamento.

Arquivos/Pastas impactados:

- app/domain/errors.py
- app/domain/entities/*
- app/domain/ports/*

Prompt para o agente (copiar e colar):

Crie o domínio do serviço:

- app/domain/errors.py: defina exceções específicas (ValidationError, DBError, StorageError, ...)
- app/domain/entities/: crie dataclasses ou pydantic models para:
 - SpreadsheetUpload (id, filename, content_type, size_bytes, uploaded_at)
 - MachineRiskRow (campos normalizados da planilha: area, equipamento, perigo, causa, consequencia)
 - ReportDraft (company/site metadata + lista MachineRiskRow + anexos)
 - GeneratedReport (report_id, pdf_path/url, created_at, checksum, version)
- app/domain/ports/: crie Protocols (typing.Protocol) para:
 - SpreadsheetParserPort.parse(file_bytes, filename) -> list[MachineRiskRow]
 - SpreadsheetValidatorPort.validate(rows) -> None (lança ValidationError)
 - ReportRepositoryPort.save_draft / save_generated / get
 - StoragePort.put/get/signed_url
 - LLMPort.generate_sections(context) -> dict
 - PdfRendererPort.render(html, assets) -> bytes
 - NormsProviderPort.get_rules(context) -> dict

Foque em contratos claros e tipos.

Etapa 4 — Adapter de planilha: parsing e normalização

Objetivo: Converter XLSX/CSV para lista de MachineRiskRow, com aliases de colunas e normalização de strings/vazios.

Arquivos/Pastas impactados:

- app/adapters/spreadsheet/parser.py
- tests/test_parser.py

Prompt para o agente (copiar e colar):

Implemente o adapter de parsing de planilha:

Arquivo: app/adapters/spreadsheet/parser.py

- Classe PandasSpreadsheetParser que implementa SpreadsheetParserPort.
- Aceite .xlsx e .csv por filename.
- Use pandas.read_excel / read_csv.
- Faça normalização:
 - trim em strings
 - valores vazios -> None
 - renomeie colunas por um dicionário de aliases (ex: "Equipamento", "Máquina", "Machine" -> "Equipment")
- Converta cada linha em MachineRiskRow.
- Se colunas obrigatórias faltarem, lance ValidationError informando quais.
- Inclua testes rápidos em tests/test_parser.py com um DataFrame mínimo (sem arquivo real, use dados fictícios).

Etapa 5 — Adapter de validação determinística

Objetivo: Validar regras objetivas: campos obrigatórios, domínio de risco, padrões mínimos. Retornar lista de erros com row_index/field/message.

Arquivos/Pastas impactados:

- app/adapters/spreadsheet/validator.py
- tests/test_validator.py

Prompt para o agente (copiar e colar):

Implemente validação determinística:

Arquivo: app/adapters/spreadsheet/validator.py

- Classe BasicSpreadsheetValidator implementando SpreadsheetValidatorPort.

- Recebe list[MachineRiskRow] e valida:

- não pode ser lista vazia

- campos obrigatórios não vazios (equipamento, perigo, causa, consequencia, risco)

- risco deve estar em conjunto permitido (case-insensitive)

- se existir norma_ref, validar padrão mínimo (ex: "ABNT", "NBR", etc) sem ser rígido demais

- Em caso de erros, lance ValidationError contendo:

- lista de erros com {row_index, field, message}

- Escreva tests/test_validator.py cobrindo 3 casos: ok, faltando campo, risco inválido.

Etapa 6 — Persistência (Postgres/Supabase)

Objetivo: Criar models/tabelas e repositório. Persistir upload, draft (rows jsonb) e report gerado.

Arquivos/Pastas impactados:

- app/infrastructure/db.py
- app/adapters/db/models.py
- app/adapters/db/repository.py
- app/scripts/init_db.py

Prompt para o agente (copiar e colar):

Implemente persistência Postgres (Supabase):

Use SQLAlchemy 2.0 async + asyncpg.

Crie:

- app/infrastructure/db.py: cria AsyncEngine e sessionmaker; função get_session().
- app/adapters/db/models.py: defina tabelas:
uploads(id uuid, filename, content_type, size_bytes, storage_path, created_at)
report_drafts(id uuid, upload_id, metadata jsonb, rows jsonb, created_at)
generated_reports(id uuid, draft_id, pdf_storage_path, pdf_url, checksum, version int, created_at)
- app/adapters/db/repository.py: ReportRepository implementando ReportRepositoryPort com métodos:
create_upload(...)
create_draft(...)
create_generated(...)
get_draft(id)
get_generated(id)

Inclua um script simples app/scripts/init_db.py para criar tabelas via metadata.create_all (a
Não use alembic ainda (deixe preparado).

Etapa 7 — Storage (Supabase): upload + clone + PDF

Objetivo: Implementar StoragePort para subir bytes e gerar signed URLs. Expiração: salvar expires_at no DB + cleanup job (não confiar em expiração automática).

Arquivos/Pastas impactados:

- app/adapters/storage/supabase_storage.py
- app/application/use_cases/cleanup_expired.py

Prompt para o agente (copiar e colar):

Implemente Storage com Supabase:

Arquivo: app/adapters/storage/supabase_storage.py

- Use supabase-py (se não estiver no requirements, adicione) OU use chamadas HTTP via httpx.
- Implementar StoragePort:
put_bytes(bucket, path, data: bytes, content_type: str, metadata: dict) -> str (retorna path)
get_signed_url(bucket, path, expires_seconds: int) -> str

Crie convenções de paths:

```
uploads/{upload_id}/{filename}
uploads/{upload_id}/clone_{YYYYMMDD}_{filename}
reports/{report_id}/report_v{version}.pdf
```

Expiração 6 meses:

- Salve expires_at no DB (tabela uploads e generated_reports).
 - Crie app/application/use_cases/cleanup_expired.py (placeholder) que futuramente remove do
- Não tente fazer “expiração mágica” no storage.

Etapa 8 — LLM (OpenRouter): gerar seções em JSON

Objetivo: Client HTTP async com retries, timeout e validação estrita de JSON. LLM só para texto (recomendações/justificativas/sumário).

Arquivos/Pastas impactados:

- app/adapters/llm/openrouter_client.py
- app/adapters/llm/prompts.py

Prompt para o agente (copiar e colar):

Implemente client OpenRouter:

Arquivo: app/adapters/llm/openrouter_client.py

- Use httpx AsyncClient
- Endpoint OpenRouter /chat/completions
- Settings: OPENROUTER_API_KEY, OPENROUTER_BASE_URL, LLM_MODEL
- Método: generate_sections(context) -> dict
 - Forçar saída JSON (ex: {"sumario": "...", "recomendacoes":[...], "justificativas":[...]})
 - Validar JSON; se falhar, tentar 1 retry com instrução de "apenas JSON"
- Use tenacity para retry em timeout/429/5xx
- Logging do request (sem vaziar chave)

Crie também app/adapters/llm/prompts.py com:

- system prompt fixo
- user prompt template

Etapa 9 — Template HTML + render PDF (WeasyPrint)

Objetivo: Renderizar HTML determinístico com Jinja2 e converter para PDF. Separar templates e CSS. Criar script local de geração para teste.

Arquivos/Pastas impactados:

- app/adapters/pdf/renderer.py
- app/adapters/pdf/templates/report.html
- app/adapters/pdf/templates/styles.css
- app/scripts/generate_sample_pdf.py

Prompt para o agente (copiar e colar):

Implemente geração de PDF:

```
- app/adapters/pdf/renderer.py
  Classe WeasyPdfRenderer implements PdfRendererPort:
    render(html: str, assets: dict) -> bytes

- app/adapters/pdf/templates/report.html (Jinja2)
  Deve receber:
    metadata (empresa, unidade, data, responsavel, etc)
    rows (lista MachineRiskRow)
    llm_sections (dict)
  Estrutura sugerida:
    capa
    sumário
    introdução
    metodologia
    tabela principal (linhas)
    recomendações e justificativas (LLM)
    referências

- app/adapters/pdf/templates/styles.css
  Defina margens e tipografia consistente, títulos, page breaks.
  Inclua numeração de página.
```

Crie um script app/scripts/generate_sample_pdf.py que cria um PDF com dados fake e salva localmente.

Etapa 10 — Use Case central: ProcessUploadAndGenerateReport

Objetivo: Orquestrar pipeline completo: storage upload + parse + validate + draft + llm + template + pdf + storage pdf + db report. Calcular checksum e versionamento.

Arquivos/Pastas impactados:

- app/application/use_cases/process_upload.py

Prompt para o agente (copiar e colar):

Implemente o caso de uso principal:

Arquivo: app/application/use_cases/process_upload.py

Crie uma classe ProcessUploadUseCase com método async execute(file_bytes, filename, content_type):

- upload_id
- draft_id
- report_id
- pdf_url

Fluxo:

- 1) repository.create_upload + storage.put_bytes para uploads/{upload_id}/{filename}
- 2) parser.parse(file_bytes, filename)
- 3) validator.validate(rows)
- 4) repository.create_draft(upload_id, metadata, rows)
- 5) llm.generate_sections(context) (context baseado no draft)
- 6) jinja2 render report.html + styles.css
- 7) pdf_renderer.render(html) -> bytes
- 8) storage.put_bytes para reports/{report_id}/report_v1.pdf
- 9) repository.create_generated(draft_id, path, url, checksum, version=1)

Inclua checksum sha256 do PDF.
Use exceções do domain e log adequado.

Etapa 11 — Rotas FastAPI (upload e consulta)

Objetivo: Criar endpoints para o front: POST /uploads (multipart), GET /reports/{id}, GET /health. Rotas devem ser finas e usar DI para obter use cases.

Arquivos/Pastas impactados:

- app/api/main.py
- app/api/routes/uploads.py
- app/api/routes/documents.py
- app/api/routes/health.py
- app/infrastructure/dependencies.py

Prompt para o agente (copiar e colar):

Implemente rotas FastAPI:

```
- app/api/main.py cria app e inclui routers.
- app/api/routes/health.py: GET /health
- app/api/routes/uploads.py:
  POST /uploads
  - recebe UploadFile
  - lê bytes
  - chama ProcessUploadUseCase
  - retorna JSON com ids e pdf_url

- app/api/routes/documents.py:
  GET /reports/{report_id}
  - busca no repository e retorna
```

Implemente injeção de dependências em app/infrastructure/dependencies.py:

```
- get_parser(), get_validator(), get_repo(), get_storage(), get_llm(), get_pdf_renderer(), ge
```

Não deixe segredos em logs.

Etapa 12 — Qualidade mínima: testes, scripts e README

Objetivo: Cobrir parser/validator e happy path do use case com mocks. Documentar como rodar local e configurar .env.

Arquivos/Pastas impactados:

- tests/*
- README.md
- requirements.txt (pytest)
- app/scripts/*

Prompt para o agente (copiar e colar):

```
Finalize com qualidade mínima:

- Adicione pytest e pytest-asyncio ao requirements.txt
- Crie tests/:
  test_parser.py
  test_validator.py
  test_usecase_happy_path.py (mockando storage e llm)

- Crie README.md com:
  - como configurar .env
  - como criar venv e instalar deps
  - como rodar uvicorn
  - como inicializar db local (create_all)
  - como testar endpoint /uploads
```

Mantenha simples e executável.

Checklist final (antes de integrar no Next)

- POST /uploads retorna erros de validação com row_index/field/message.
- Draft e Report ficam versionados no DB (upload → draft → report).
- Storage não sobrescreve arquivos; sempre novo path.
- LLM responde JSON validado; se falhar, retry + erro claro.
- Script local gera PDF sample sem depender do front.
- Testes rodam com mocks para Storage e LLM.

Sugestão de comando de execução local:

```
(.venv) pip install -r requirements.txt  
(.venv) python -m app.scripts.init_db  
(.venv) uvicorn app.api.main:app --reload --port 8001
```